

Kaybolan Gerekçe: Devir Teslimsiz Kod Tabanlarında Çalışma

ÖFY

August 2026

Özet

Bir yazılım projesinde devir teslim yapılmadığında kaybolan şey bilgi değil, gerekçedir. Kaynak kodu neyin yapıldığını taşır; e-posta yazışmaları, toplantılar ve karar süreçleri ise neyin yapılmadığını ve neden yapılmadığını taşır. Bu ikinci küme hiçbir teknik artefakta yazılmadığı için kişiyle birlikte kaybolur. Devralan kişinin yüksek oranda hata üretmesinin mekanizması budur: gerekçesi görünmeyen bir yapı, gereksiz karmaşıklık olarak okunur ve sadeleştirilir; sadeleştirmenin ihlal ettiği kısıt bir süre sonra üretimde ortaya çıkar. Bilginin saklandığı ortamlar, ihlal edildiğinde kendini duyurma kapasitesine göre sıralanabilir ve bu sıralamada dokümantasyon zayıf, çalışma anı kontrolleri güçlü uçtadır. Devir teslimin bulunmadığı bir ortamda tek işleyen strateji, kritik kısıtları okunması gereken biçimlerden ihlal edildiğinde sinyal üreten biçimlere taşımaktır. Devralan taraf için ise kaybolan spesifikasyonun yerine sistemin mevcut davranışı konur: gerekçe geri kazanılamasa da ihlal tespit edilebilir hâle getirilebilir. Büyük dil modelleri bu tabloyu iyileştirmez, çünkü model tam olarak kodu okuyup gerekçeyi göremeyen okuyucudur ve üretim hızını artırırken doğrulama hızını artırmaz.

Giriş

Bir yazılım sisteminin kaynak kodu, o sistemin nasıl davrandığının eksiksiz bir kayıdır. Buna karşılık sistemin neden öyle davrandığının kaydı değildir. Bir fonksiyonun içindeki alışılmadık bir dönüşüm, bir koşul bloğu ya da görünürde gereksiz bir bekleme, kodda yalnızca varlığıyla görünür; varlık sebebi kodun dışında bir yerde kalmıştır.

Bu ayrım için **gereke** (*rationale*) terimi kullanılacaktır: bir tasarım kararının hangi kısıtlar, hangi alternatifler ve hangi olaylar sonucunda alındığı bilgisi. Gereke tipik olarak e-posta yazışmalarında, toplantı kayıtlarında, müşteri görüşmelerinde ve kişilerin hafızasında bulunur. Bunların hiçbirisi kaynak kod deposunun parçası değildir.

Devir teslim (*handover*), bir projede sorumluluğu bırakan kişinin biriktirdiği bağlamı devralan kişiye aktardığı süreçtir. Kurumsal pratikte bu süreç çoğu zaman ya hiç yapılmaz ya da yalnızca kodun ve çalıştırma yordamlarının aktarımıyla sınırlı kalır. Gereke aktarılmaz, çünkü gerekçenin kendisi çoğu zaman aktaran kişide de açık biçimde durmaz; yıllar içinde sezgiye dönüşmüş, kaynağı unutulmuş bir “şu böyle olmaz” bilgisi hâline gelmiştir.

Naur (1985), programlamanın asıl ürününün kaynak kodu değil, programcılarının zihninde kurulan *kuram* olduğunu ileri sürmüştür. Bu görüşe göre bir program, onu üretmiş ekip dağıldığında teknik olarak çalışmaya devam etse bile kavramsal olarak ölmüştür; çünkü programın hangi gerçek dünya olgusuna karşılık geldiği bilgisi metne yazılamaz. Buradaki problem bu tespitin pratik sonucudur.

Yaygın kabul, bu problemin çözümünün dokümantasyon olduğu yönündedir. Bu kabul kısmen yanlıştır ve yanlışlık şu noktadadır: devir tesliminin yapılmadığı bir kurumda dokümanın okunacağını varsaymak için bir sebep yoktur. Dokümanı okumayı gerektiren aynı kurumsal disiplin, devir teslimi de gerektirirdi. Dolayısıyla dokümantasyon, çözümün tamamı değil zayıf bir bileşenidir.

Burada ele alınan kapsam iki taraflıdır: hâlen sistem üzerinde çalışan kişinin kendinden sonrası için yapabilecekleri ve hiçbir bağlam devralmamış kişinin hata üretmeden çalışabilmek için yapabilecekleri. Kapsam dışında kalan konu, kurumsal düzeyde bilgi yönetimi politikalarının tasarımıdır; burada varsayılan, bireyin böyle bir politikayı dayatma yetkisinin bulunmadığıdır.

Bilginin Saklandığı Ortam ve Hayatta Kalma

Bir kısıt bilgisinin projede kalıcı olup olmaması, o bilginin doğruluğuyla değil hangi ortamda saklandığıyla ilgilidir. Ortamlar, devir teslim gerçekleşmediğinde bilgiyi koruma kapasitelerine göre sıralanabilir.

insanın kafası	→ kişi giderse yok olur
e-posta / toplantı	→ arşivde durur ama kimse aramaz
wiki / doküman	→ bulunur ama güncelliği doğrulanmaz
kod yorumu	→ kodla birlikte okunur, uyulmasa da sessizdir
tip sistemi	→ uyulmazsa derlenmez
test	→ uyulmazsa kırmızı yanar
çalışma anı kontrolü	→ uyulmazsa üretimde bağırır

Bu sıralamayı üreten ölçüt bilginin miktarı ya da ayrıntısı değildir. Ölçüt şudur: bilgi ihlal edildiğinde kendiliğinden bir sinyal üretiyor mu. Üst sıralardaki ortamlarda bilgi pasiftir; birinin onu arayıp bulması, okuması ve doğru yorumlaması gerekir. Alt sıralarda bilgi aktiftir; ihlal edildiği anda bir derleme hatası, bir başarısız test ya da bir uyarı kaydı üretir.

Ayrımın pratik önemi şuradadır. Üst sıradaki bir ortama yazılmış kısıt, yalnızca o kısıtın var olduğunu *zaten tahmin eden* bir okuyucuya ulaşır; çünkü aramayı ancak bir şeyin arandığını bilen kişi yapar. Bağlamı olmayan yeni kişinin problemi ise tam olarak neyi arayacağını bilmemesidir. Dolayısıyla dokümantasyon, en çok ihtiyaç duyulduğu durumda en az işe yarayan çözümdür.

Alt sıralardaki ortamlar bu bağımlılığı ortadan kaldırır. Bir kısıt tip sistemine yazıldığında, onu ihlal eden kişinin kısıttan haberdar olması gerekmez; derleyici haberdar eder. Bu, bilginin aktarılması değil, aktarılma ihtiyacının ortadan kaldırılmasıdır.

Sıralamanın sınırları

Bu sıralama mutlak bir üstünlük düzeni değildir. Her kısıt alt sıralara taşınamaz: “müşteri bu ekranı 2019’daki bir şikâyet üzerine istedi” bilgisi bir teste dönüştürülemez, ancak bir karar kaydında saklanabilir. Benzer biçimde, bir testin ifade ettiği kısıt testin kendisinde okunabilir hâlde değildir; test kısıtın ihlal edildiğini bildirir ama neden var olduğunu söylemez.

Bu nedenle doğru strateji, üst sıraları terk etmek değil, kritik olan az sayıda kısıtı alt sıralara indirip geri kalanı yazılı biçimde tutmaktır. Kritiklik ölçütü şudur: ihlal edildiğinde sessizce yanlış davranış üreten kısıtlar önceliklidir, çünkü bunlar zaten hiçbir sinyal üretmeyen sınıftır.

Üreten Taraf: Gerekçeyi Koda Gömmek

Hâlen sistem üzerinde çalışan kişinin yapabilecekleri, kendi dokunduğu yüzeyle sınırlıdır ancak etkisizdir demek değildir.

Her hatayı bir teste çevirmek

Bir hata (*bug*), tanım gereği kaybolmuş bir kısıtın kanıtıdır: sistemin uyması gereken ama uymadığı bir koşul bulunmaktadır. Bu durumda iki iş vardır; hatanın düzeltilmesi ve aynı hatanın aynı yerden bir daha çıkmasını imkânsız kılan bir testin yazılması.

İkincisi uzun vadede daha değerlidir. Düzeltme yalnızca mevcut durumu onarır; test ise kaybolmuş kısıtı geri kazanıp koda gömer. Bir yıl sonra o testi kıran kişi, hiç haberdar olmadığı bir kısıtla karşılaşmış olur ve bunu üretim ortamında değil kendi makinesinde öğrenir. Bu tür testler **regresyon testi** (*regression test*) olarak adlandırılır; işlevi yeni davranışı doğrulamak değil, eski bir arızanın geri dönmediğini garanti etmektir.

Sınır noktalarına açık kontrol koymak

Sistemin dış dünyayla ve modüllerin birbiriyle temas ettiği noktalarda, örtük varsayımlar açık kontrollere çevrilir. “Bu alan hiçbir zaman boş gelmez” bilgisi bir kişinin hafızasında bulunuyorsa kaybolur; bir koşul bloğuna yazılıp ihlali kayda düşürüldüğünde kaybolmaz.

Bu dönüşümün ikinci bir kazancı vardır: kontrol ihlal edildiği gün, hangi çağrının hangi veriyle geldiği kayıta görünür. Yani kısıt yalnızca korunmaz, ihlal anında gerekçesini yeniden üretecek veri de toplanmış olur.

“Neden” yorumları yazmak

Kodun ne yaptığını kodun kendisi söyler; bunu tekrar eden yorum, kod değiştiğinde güncellenmediği için zamanla yanlış bilgiye dönüşür. Yorumun tek meşru işlevi, koddan okunamayan gerekçeyi taşımaktır.

```
// KÖTÜ
// Tarihi ISO formatına çevirir.

// İYİ
// Burada ekstra dönüşüm var çünkü X sistemi milisaniye alanı
// içeren ISO damgalarını reddediyor (2023 entegrasyon arızası).
```

İkinci örnekteki tek cümle, kaybolmuş bir toplantının yerine geçer. Bu yorumu okuyan kişi dönüşümü kaldırmaz.

Mimari karar kayıtları

Mimari karar kaydı (*architecture decision record*, ADR), önemli bir tasarım kararını dört başlıkla saklayan kısa bir metin dosyasıdır: bağlam, verilen karar, kararın sonuçları ve reddedilen alternatifler (Nygard, 2011). Kod deposunun içinde, kaynak dosyalarla aynı yerde tutulur; böylece kodla birlikte sürümlenir ve kod incelemesi sırasında görülür.

Bu kayıtların değeri asıl olarak reddedilen alternatifler bölümündedir. Devralan kişinin ilk yapmak isteyeceği değişiklik, çoğu zaman bir zamanlar denenip elenmiş olan şeydir; elenme sebebinin yazılı olması bu tekrarı önler.

Uzunluk kısıtı işlevseldir: on satırı aşan kayıtlar yazılmaz, bir sayfayı aşanlar okunmaz. Kaydın eksiksiz olması değil, var olması belirleyicidir.

Var olan bağlamı çıkarmak

Kaybolduğu düşünülen bağlamın bir kısmı hâlen depoda durmaktadır: işleme (*commit*) mesajları, birleştirme istekleri (*pull request*) üzerindeki tartışmalar ve iş kaydı (*issue*) geçmişi. Bu kaynak sistematik olarak yeterince kullanılmaz, çünkü aranabilir olduğu akla gelmez.

Devralan Taraf: Bağımsız Kod Tabanında Çalışma

Hiçbir karar kaydı, doküman ya da devir teslim almamış kişinin elinde tek bir kaynak vardır: çalışan sistemin kendisi. Gerekçe kaybolmuş olabilir ancak davranış durmaktadır. Strateji buradan türetilir: kaybolan spesifikasyonun yerine mevcut davranış konur, değişiklik ondan sonra yapılır.

Davranışı dondurmak

Anlaşılmayan bir kod parçası değiştirilmeden önce, o parçanın hâlihazırda ne yaptığını yakalayan bir test yazılır. Doğru olduğu düşünülen davranış değil, fiilen gerçekleşen davranış test edilir; çıktı hatalı görünse bile test o çıktıyı bekler. Bu tür testlere **karakterize edici test** (*characterization test*) denir (Feathers, 2004).

Mantığı şudur: kaybolan gerekçe, bir noktada bir davranışa dönüşmüş durumdadır. O davranış test altına alındığında, gerekçe bilinmese de ihlal edildiğinde sinyal üretilir. Burada bilgi geri kazanılmamakta, ihlal tespit edilebilir hâle getirilmektedir; pratik koruma açısından ikisi denktir.

Uygulama kaba olabilir. Fonksiyonun gerçek girdilerle ürettiği çıktı bir dosyaya kaydedilir ve test bu dosya ile karşılaştırma yapar; bu yöntem **altın kopya** (*golden master*) testi denir. Zarif bir çözüm değildir, ancak birkaç saat içinde kurulum ve en sık kırılan bölgeyi kapatır.

Depodaki tarihsel izi okumak

Sürüm kontrol sistemi, kaybolmuş yazışmaların bir kısmının izini taşır.

```
git log -S"tuhaf_gorunen_ifade" → ifadenin hangi commit'te girdiğini bulur
git log --follow -p <dosya> → dosyanın tüm değişim geçmişi
git blame <dosya> → satır satır kim, ne zaman
```

Özellikle tuhaf görünen tek satırlık düzeltmeler — bir koşul, bir bekleme çağrısı, bir özel durum kontrolü — neredeyse her zaman yaşanmış bir üretim arızasının izidir. İşleme mesajı yetersiz olsa bile tarihine bakılıp o dönemdeki diğer değişikliklerle birlikte okunması ipucu verir.

Testleri spesifikasyon olarak okumak

Var olan testler, kaybolmuş gerekçenin yazıya geçmiş tek parçasıdır. Genel testler az bilgi taşır; asıl bilgi tuhaf, spesifik ve sabit sayı içeren testlerdedir. “Şu alan boşken şu hata kodu dönmeli” biçimindeki bir test, bir zamanlar bu durumun yaşandığının kanıtıdır. Yeni bir modüle girerken önce testlerin okunması, kodun okunmasından daha hızlı bağlam sağlar.

Üretim verisine bakmak

Kodun izin verdiği durum kümesi ile üretimde fiilen oluşan durum kümesi farklıdır. Bir alan boş değer alabilir biçimde tanımlanmış olsa bile üretimde hiç boş gelmiyorsa, sistemin gerçek sözleşmesi tip tanımından dardır. Tersine de görülür: kod tek bir biçim varsayarken üretimde üç farklı biçim akmakta ve fark bir yerde sessizce yutulmaktadır.

Kayıtlar ve veritabanı, yazılı olmayan kısıtların en dürüst kaynağıdır; çünkü niyeti değil gerçekleşeni gösterirler. Bir modüle dokunmadan önce oraya giren gerçek verinin bir süre gözlenmesi, dokümandan daha fazla bilgi verir.

Yayılma alanını önceden çıkarmak

Bir birim değiştirilmeden önce onu kullanan tüm çağrı noktaları bulunur. Derleyicinin yakaladığı çağrılar problemin kolay kısmıdır; asıl risk derleyicinin göremediği bağlardadır:

- yansıma (*reflection*) ve dinamik çağrı
- yapılandırma dosyasından okunan sınıf ya da alan adları
- metin olarak yazılmış veritabanı sorguları, kolon ve tablo adları
- olay ve mesaj adları, kuyruk anahtarları
- dış sistemlerin beklediği alan adları ve biçimler

Bu beş kategori, bağlamı olmayan kişinin en sık arıza ürettiği yerlerdir; ortak özellikleri hiçbirinde aracın uyarı vermemesidir. Değişiklik öncesinde ilgili adların kod tabanında düz metin olarak aranması zorunlu adımdır.

Değiştirmek yerine eklemek

Anlaşılmayan bir yolun yerine yenisini koymak yerine, yeni yol mevcut olanın yanına eklenir ve eski yol yerinde bırakılır. İki yol bir süre birlikte çalıştırılır, çıktıları karşılaştırılır ve uyum doğrulandıktan sonra eski yol kaldırılır. Bu yöntem tek seferde değiştirmeye göre belirgin biçimde yavaştır; buna karşılık her aşamasında geri dönebilir. Gerekeceği bilinmeyen bir sistemde geri dönebilme, hızlı ilerlemeden değerlidir.

Güvenli olmak yerine hızlı fark etmek

Bağlam yokken her değişikliği güvenli hâle getirmek mümkün değildir. Bu durumda hedef değişir: değişikliğin yanlış olduğunun saatler içinde öğrenilmesi hedeflenir. Bunu sağlayan mekanizmalar küçük ve tek amaçlı değişiklikler, sık ve geri alınabilir konuşlandırma, değişen yola konmuş kayıt satırları ve değişimden sonraki ilk gün ölçütlerin izlenmesidir.

İlkesel ifadesi şudur: kaybolan bilginin telafisi doğru tahmin etmek değil, yanlış tahminin ucuz olmasıdır.

Anlařılmayanı silmemek

Gerekçesi bulunamayan bir kontrol, gereksiz olduđu kanıtlanmış bir kontrol değildir; yalnızca gerekçesi bulunamamış bir kontroldür. Bu ayrım, Chesterton (1929) tarafından farklı bir bağlamda kurulan ilkenin yazılım karşılığıdır: bir çitin neden dikildiğı bilinmeden kaldırılmasına karar verilemez.

Silme yerine izlenecek yol, ilgili noktaya kayıt satırı konup bir süre beklenmesidir. Kontrol hiç tetiklenmiyorsa silme kararı için dayanak oluşur; tetikleniyorsa kaybolmuş gerekçe bulunmuş olur. İki günlük bir bekleme, bu hata sınıfının tamamını önler.

Kalan insan hafızasını toplamak

Doküman bulunmasa da insanlar tamamen ayrılmamış olabilir. Destek ekibi, ürün tarafı, operasyon ve müşteriyle görüşen kişiler kodu bilmezler; ancak bir ekranın neden öyle çalıştığı sorusunun cevabını çoğı zaman bilirler, çünkü o kararın müşteri tarafındaki sebebini yaşamışlardır.

Sorunun biçimi belirleyicidir. “Bu kod ne yapıyor” sorusu bu kişilere yöneltilemez. Yöneltilebilecek soru şudur: bu böyle çalışmasaydı kim şikâyet ederdi. On dakikalık bir görüşme, iki günlük kod okumasının yerini tutabilir.

Büyük Dil Modellerinin Etkisi

Büyük dil modeli (*large language model*, LLM), bu problemde nötr bir araç değildir; mevcut asimetriyi belirli bir yönde büyütür.

Neden durumu kötüleştirir

Model, tanım gereği kodu okuyup gerekçeyi göremeyen okuyucudur. Bağlamına verilmemiş bir kısıt onun için mevcut değildir ve modelin bu kısıtı tahmin etmesi beklenemez. Dolayısıyla tuhaf duran bir yapıyı gereksiz karmaşıklık olarak yorumlaması, bir hata değil beklenen davranıştır.

İkinci ve daha önemli etki hız üzerindedir. Model üretim tarafını hızlandırır, doğrulama tarafını aynı oranda hızlandırmaz. Bir sistemde üretim kapasitesi artıp doğrulama kapasitesi sabit kaldığında darboğaz doğrulamaya kayar ve birim zamanda üretime sızan hatalı değişiklik sayısı artar. Bu, model kalitesinden bağımsız yapısal bir sonuçtur.

Kısıt dosyası

Depo içinde tutulan ve modelin bağlamına otomatik olarak giren bir kısıt dosyası — bu sistemde asla yapılmayacak işlemler, zorunlu biçim kuralları, dış sistemlerin beklentileri — çıktı kalitesini dokümantasyondan daha fazla artırır. Sebep basittir: bu dosya her istekte okunur, doküman okunmaz. Bilginin “okunması gereken” sınıftan “otomatik ulaşan” sınıfa taşınmasının bir örneğidir.

Doğrulama tarafında kullanım

Bağlamı olmayan bir kod tabanında modele temizleme, sadeleştirme ya da modernleştirme yaptırmak en riskli kullanım biçimidir. Verimli kullanım doğrulama tarafındadır:

- değişiklik öncesinde karakterize edici testlerin ürettirilmesi
- bir birimin değiştirilmesi hâlinde etkilenecek çağrı noktalarının taratılması
- hazırlanan değişikliğin gözden geçirilip doğrulanmamış varsayımların sorulması
- anlaşılmayan bir bloğun ne yaptığının açıklattırılması

Son maddede bir ayırım korunmalıdır: modelin bir kod bloğunun *ne yaptığına* dair açıklaması doğrulanabilir bir iddiadır; *neden orada olduğuna* dair açıklaması ise bir tahmindir ve kanıt sayılamaz.

Sonuç

Bağımsız bir kod tabanında hata üretmemenin yolu, kaybolan bilgiyi geri kazanmak değildir; çoğu durumda bu mümkün değildir. Yol, kaybolan bilginin yerine sistemin mevcut davranışını koymak ve her değişikliği ihlalin kendini göstereceği biçimde yapmaktır. Bilinmeyen kısıt varlığını sürdürmektedir; ihtiyaç duyulan şey onu öğrenmek değil, çığnendiğinde bunu üretim ortamından önce ya da hemen sonra duymaktır.

Bu çerçevenin işaret ettiği daha genel sonuç, dokümantasyonun bir bilgi aktarma aracı değil bir bilgi saklama aracı olduğudur. Aktarma, ancak bilgi ihlal edildiğinde sinyal üreten bir ortama taşındığında gerçekleşir. Bu nedenle “daha iyi doküman yazmak” önerisi, devir tesliminin yapılmadığı ortamlarda problemin çözümü değil, çözümün zayıf bir bileşenidir.

Çerçevenin sınırları belirtilmelidir. Karakterize edici testler mevcut davranışı korur, ancak mevcut davranış hatalıysa hatayı da korur; bu yöntem doğruluğu değil değişmezliği garanti eder. Benzer biçimde, her kısıt teste ya da tip sistemine indirilemez; müşteri tarafındaki tarihsel sebepler yalnızca yazılı biçimde saklanabilir. Paralel çalıştırma yöntemi yan etkisiz işlemlerde uygulanabilir, yan etkili işlemlerde ek düzenek gerektirir.

Açık kalan soru, bu pratiklerin bireysel düzeyde uygulanmasının kurumsal etkisiyle ilgilidir. Sayılan tedbirlerin bir kısmı — gerekçe yorumları, regresyon testleri, sınır kontrolleri, kişisel karar kayıtları, model için kısıt dosyası — tek kişinin yetkisi dahilindedir. Bir kısmı — gözden geçirme kuralları, test altyapısına ayrılan zaman, devir teslim süreci — takım kararı gerektirir. İkinci grup yokken birinci grup yine işe yarar, ancak etkisi kişinin dokunduğu yüzeyle sınırlı kalır. Kurumsal hafıza tek kişi tarafından kurulamaz; buna karşılık tek kişi, kendi dokunduğu yüzeyi bir sonrakinin gerekçe aramak zorunda kalmayacağı hâle getirebilir.

Kaynaklar

- Brooks, F. P. (1975). *The Mythical Man-Month*. Addison-Wesley.
- Chesterton, G. K. (1929). *The Thing: Why I Am a Catholic*. Dodd, Mead & Company.
- Feathers, M. (2004). *Working Effectively with Legacy Code*. Prentice Hall.
- Naur, P. (1985). Programming as Theory Building. *Microprocessing and Microprogramming*, 15(5).
- Nygard, M. (2011). Documenting Architecture Decisions. *Cognitect Blog*.
- Parnas, D. L. & Clements, P. C. (1986). A Rational Design Process: How and Why to Fake It. *IEEE Transactions on Software Engineering*, SE-12(2).